Backpropagation krok po kroku
Mały przykład sieci neuronowej

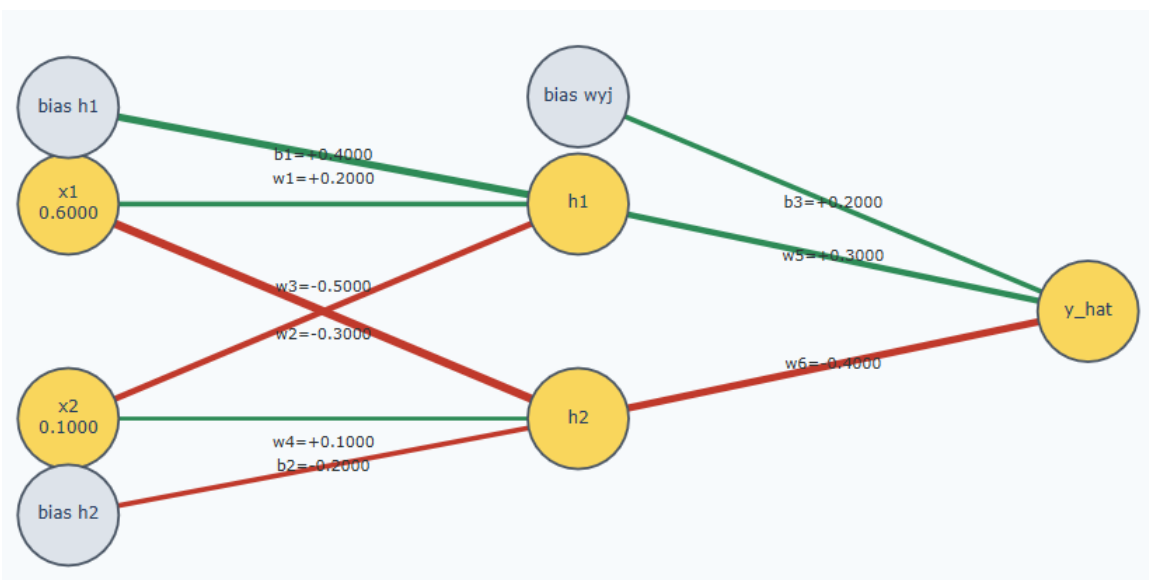
Grzegorz Madejski

1 Wprowadzenie i struktura sieci

Rozważmy bardzo małą sieć neuronową:

- 2 neurony wejściowe,
- 2 neurony w warstwie ukrytej,
- 1 neuron wyjściowy.

Neurony w warstwie ukrytej używają funkcji aktywacji **sigmoid**. Neuron wyjściowy jest liniowy (dla uproszczenia rachunków).



Niech wejście będzie równe

$$x_1 = 0.6, \qquad x_2 = 0.1$$

a prawdziwa wartość wyjścia (target) wynosi

$$y = 0.8.$$

2 Parametry sieci

2.1 Wagi i biasy między wejściem a warstwą ukrytą

Dla neuronu h_1:

$$w_1 = 0.2, \qquad w_2 = -0.3, \qquad b_1 = 0.4$$

Dla neuronu h_2:

$$w_3 = -0.5, \qquad w_4 = 0.1, \qquad b_2 = -0.2$$

2.2 Wagi i bias neuronu wyjściowego

$$w_5 = 0.3, \qquad w_6 = -0.4, \qquad b_3 = 0.2$$

3 Funkcja aktywacji

W warstwie ukrytej używamy funkcji sigmoidalnej:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Jej pochodna ma bardzo wygodną postać:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

To właśnie ta pochodna będzie potrzebna podczas backpropagation.

4 Propagacja w przód

W tej części obliczamy wyjście sieci dla danego przykładu wejściowego.

4.1 Obliczenie wartości neuronu h_1

Najpierw liczymy sumę ważoną:

$$z_1 = w_1 x_1 + w_2 x_2 + b_1$$

Po podstawieniu:

$$z_1 = (0.2)(0.6) + (-0.3)(0.1) + 0.4$$

$$z_1 = 0.12 - 0.03 + 0.4 = 0.49$$

Następnie stosujemy funkcję sigmoid:

$$h_1 = \sigma(z_1) = \sigma(0.49) \approx 0.6201$$

4.2 Obliczenie wartości neuronu h_2

Liczymy sumę ważoną:

$$z_2 = w_3 x_1 + w_4 x_2 + b_2$$

Po podstawieniu:

$$z_2 = (-0.5)(0.6) + (0.1)(0.1) - 0.2$$

$$z_2 = -0.3 + 0.01 - 0.2 = -0.49$$

Stąd:

$$h_2 = \sigma(z_2) = \sigma(-0.49) \approx 0.3799$$

4.3 Obliczenie wyjścia sieci

Neuron wyjściowy jest liniowy, więc:

$$\hat{y} = w_5 h_1 + w_6 h_2 + b_3$$

Po podstawieniu:

$$\hat{y} = (0.3)(0.6201) + (-0.4)(0.3799) + 0.2$$

$$\hat{y} \approx 0.1860 - 0.1520 + 0.2 = 0.2341$$

Zatem predykcja sieci wynosi:

$$\hat{y} \approx 0.2341$$

5 Funkcja straty

Chcemy porównać predykcję $\hat{y}$ z wartością prawdziwą $y = 0.8$.

Użyjemy błędu średniokwadratowego:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Po podstawieniu:

$$L = \frac{1}{2}(0.2341 - 0.8)^2$$

$$L = \frac{1}{2}(-0.5659)^2 \approx 0.1601$$

Im większa wartość L, tym większy błąd modelu. Celem uczenia jest takie zmienianie wag i biasów, aby strata malała.

6 Backpropagation krok po kroku

Teraz chcemy policzyć, jak zmienić wagi, aby zmniejszyć błąd.

6.1 Idea gradientu

Dla każdej wagi chcemy policzyć pochodną postaci:

$$\frac{\partial L}{\partial w}$$

Taka pochodna mówi, jak bardzo zmieni się funkcja straty, gdy lekko zmienimy wagę w.

- Jeśli pochodna jest dodatnia, to zwiększanie wagi zwiększa stratę.
- Jeśli pochodna jest ujemna, to zwiększanie wagi zmniejsza stratę.

Dlatego w metodzie gradient descent aktualizujemy parametry według wzoru:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

gdzie η oznacza współczynnik uczenia.

6.2 Dlaczego potrzebna jest reguła łańcuchowa

W sieci neuronowej waga wpływa na stratę nie bezpośrednio, lecz przez wiele kolejnych etapów.

Na przykład dla wagi w_1 zależność wygląda tak:

$$w_1 \to z_1 \to h_1 \to \hat{y} \to L$$

Dlatego używamy **reguły łańcuchowej**:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

6.3 Krok 1: błąd na wyjściu

Najpierw obliczamy tzw. **sygnał błędu** dla neuronu wyjściowego. Definiujemy go jako pochodną funkcji straty względem sumy ważonej neuronu:

$$\delta_3 = \frac{\partial L}{\partial z_3}$$

gdzie

$$z_3 = w_5 h_1 + w_6 h_2 + b_3$$

W naszym przykładzie neuron wyjściowy jest liniowy, więc

$$\hat{y} = z_3$$

Użycie reguły łańcuchowej

Ponieważ funkcja straty zależy od $\hat{y}$, a $\hat{y}$ zależy od z_3, musimy użyć reguły łańcuchowej:

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3}$$

Najpierw liczymy pierwszą pochodną:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

Ponieważ neuron wyjściowy jest liniowy (czasem jest tutaj ina funkcja np. softmax):

$$\hat{y} = z_3$$

więc

$$\frac{\partial \hat{y}}{\partial z_3} = 1$$

Po połączeniu wszystkiego:

$$\delta_3 = \frac{\partial L}{\partial z_3} = (\hat{y} - y) \cdot 1 = \hat{y} - y$$

Po podstawieniu wartości liczbowych:

$$\delta_3 = 0.2341 - 0.8 = -0.5659$$

Interpretacja sygnału błędu

Wielkość δ_3 nazywamy **sygnałem błędu neuronu**.

Można ją interpretować jako informację:

- jak bardzo neuron przyczynił się do błędu modelu,
- w którą stronę należy zmienić jego wejście z_3, aby zmniejszyć funkcję straty.

Jeśli $\delta_3 < 0$, oznacza to że wyjście sieci jest zbyt małe i zwiększenie z_3 zmniejszyłoby błąd.

Jeśli $\delta_3 > 0$, wyjście jest zbyt duże i należy je zmniejszyć.

Ten sygnał błędu jest następnie **propagowany wstecz przez sieć**, aby obliczyć błędy neuronów w warstwie ukrytej.

6.4 Krok 2: gradienty wag warstwy wyjściowej

Teraz obliczamy gradienty wag prowadzących do neuronu wyjściowego.

Przypomnijmy równanie neuronu:

$$z_3 = w_5 h_1 + w_6 h_2 + b_3$$

Ponieważ neuron wyjściowy jest liniowy, mamy

$$\hat{y} = z_3$$

Chcemy policzyć pochodne funkcji straty względem wag w_5 oraz w_6.

Użycie reguły łańcuchowej

Ponownie stosujemy regułę łańcuchową.

Dla wagi w_5:

$$\frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Pierwszy czynnik to sygnał błędu neuronu wyjściowego:

$$\frac{\partial L}{\partial z_3} = \delta_3$$

Drugi czynnik wynika bezpośrednio z równania neuronu:

$$z_3 = w_5 h_1 + w_6 h_2 + b_3$$

więc

$$\frac{\partial z_3}{\partial w_5} = h_1$$

Po połączeniu:

$$\frac{\partial L}{\partial w_5} = \delta_3 h_1$$

Analogicznie dla drugiej wagi:

$$\frac{\partial L}{\partial w_6} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_6}$$

Ponieważ

$$\frac{\partial z_3}{\partial w_6} = h_2$$

otrzymujemy

$$\frac{\partial L}{\partial w_6} = \delta_3 h_2$$

Podstawienie wartości liczbowych

$$\frac{\partial L}{\partial w_5} = (-0.5659)(0.6201) \approx -0.3509$$

$$\frac{\partial L}{\partial w_6} = (-0.5659)(0.3799) \approx -0.2150$$

Dla biasu wyjściowego:

$$\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial b_3}$$

Ponieważ

$$\frac{\partial z_3}{\partial b_3} = 1$$

otrzymujemy

$$\frac{\partial L}{\partial b_3} = \delta_3 = -0.5659$$

Dlaczego gradient ma taką postać

Warto zauważyć, że gradient każdej wagi ma prostą strukturę:

$$\text{gradient} = (\text{sygnał błędu neuronu}) \times (\text{wejście do tej wagi})$$

W tym przypadku:

- sygnałem błędu jest δ_3,
- wejściami są h_1 oraz h_2.

Nie jest to przypadek — ta struktura pojawia się w całym algorytmie backpropagation.

Czy błąd zawsze mnożymy przez wagę?

Podczas propagacji błędu do wcześniejszych warstw pojawia się mnożenie przez wagi, ale dzieje się to tylko wtedy, gdy **przenosimy wpływ błędu przez połączenie między neuronami**.

Na przykład w kolejnym kroku zobaczymy zależność:

$$\delta_1 = \delta_3 \, w_5 \, \sigma'(z_1)$$

Tutaj:

- w_5 mówi, jak silnie neuron h_1 wpływa na neuron wyjściowy,
- dlatego błąd wyjścia musi zostać przeskalowany przez tę wagę.

Można to interpretować tak:

im większa waga połączenia, tym większy wpływ neuronu na błąd końcowy modelu.

6.5 Krok 3: propagacja błędu do warstwy ukrytej

W poprzednim kroku policzyliśmy sygnał błędu neuronu wyjściowego

$$\delta_3 = \frac{\partial L}{\partial z_3}$$

Teraz chcemy policzyć błędy neuronów w warstwie ukrytej, czyli

$$\delta_1 = \frac{\partial L}{\partial z_1}, \qquad \delta_2 = \frac{\partial L}{\partial z_2}.$$

Oznacza to: jak bardzo zmiana sumy ważonej z_1 lub z_2 wpływa na funkcję straty.

Dlaczego używamy reguły łańcuchowej

Neuron ukryty nie wpływa bezpośrednio na funkcję straty. Wpływ przebiega przez kilka etapów:

$$z_1 \to h_1 \to z_3 \to L$$

Dlatego stosujemy regułę łańcuchową:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1}$$

Każdy z tych czynników ma jasną interpretację:

- $\frac{\partial L}{\partial z_3}$ — błąd na wyjściu,
- $\frac{\partial z_3}{\partial h_1}$ — jak silnie neuron h_1 wpływa na wyjście,
- $\frac{\partial h_1}{\partial z_1}$ — pochodna funkcji aktywacji.

Obliczenie błędu neuronu h_1

Wiemy że

$$z_3 = w_5 h_1 + w_6 h_2 + b_3$$

więc

$$\frac{\partial z_3}{\partial h_1} = w_5$$

Neuron ukryty używa funkcji sigmoid:

$$h_1 = \sigma(z_1)$$

więc

$$\frac{\partial h_1}{\partial z_1} = \sigma'(z_1)$$

Zatem

$$\delta_1 = \frac{\partial L}{\partial z_1} = \delta_3 \cdot w_5 \cdot \sigma'(z_1)$$

Najpierw liczymy pochodną sigmoidu:

$$\sigma'(z_1) = h_1(1 - h_1)$$

$$\sigma'(z_1) = 0.6201(1 - 0.6201) \approx 0.2356$$

Stąd

$$\delta_1 = (-0.5659)(0.3)(0.2356) \approx -0.0400$$

Obliczenie błędu neuronu h_2

Analogicznie analizujemy zależność

$$z_2 \to h_2 \to z_3 \to L$$

Stosujemy regułę łańcuchową:

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial z_2}$$

Z równania neuronu wyjściowego mamy

$$\frac{\partial z_3}{\partial h_2} = w_6$$

A pochodna sigmoidu wynosi

$$\sigma'(z_2) = h_2(1 - h_2)$$

$$\sigma'(z_2) = 0.3799(1 - 0.3799) \approx 0.2356$$

Zatem

$$\delta_2 = \delta_3 w_6 \sigma'(z_2)$$

Po podstawieniu:

$$\delta_2 = (-0.5659)(-0.4)(0.2356) \approx 0.0533$$

Interpretacja

Widzimy tutaj ważną zasadę backpropagation:

błąd jest przenoszony wstecz przez sieć i skalowany przez wagi oraz pochodne funkcji aktywacji.

- waga mówi jak silny wpływ ma neuron na kolejną warstwę,
- pochodna aktywacji mówi jak bardzo neuron reaguje na zmianę swojego wejścia.

Dlatego oba czynniki pojawiają się we wzorze na δ.

6.6 Krok 4: gradienty wag warstwy ukrytej

Teraz znamy już błędy neuronów ukrytych:

$$\delta_1 = \frac{\partial L}{\partial z_1}, \qquad \delta_2 = \frac{\partial L}{\partial z_2}.$$

Możemy więc policzyć gradienty wag prowadzących z wejścia do warstwy ukrytej.

Gradienty wag neuronu h_1

Neuron h_1 ma sumę ważoną

$$z_1 = w_1 x_1 + w_2 x_2 + b_1$$

Chcemy policzyć

$$\frac{\partial L}{\partial w_1}$$

Ponownie stosujemy regułę łańcuchową:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Pierwszy czynnik to

$$\frac{\partial L}{\partial z_1} = \delta_1$$

Drugi wynika bezpośrednio z równania neuronu:

$$\frac{\partial z_1}{\partial w_1} = x_1$$

Stąd

$$\frac{\partial L}{\partial w_1} = \delta_1 x_1$$

Analogicznie:

$$\frac{\partial L}{\partial w_2} = \delta_1 x_2$$

Po podstawieniu wartości liczbowych:

$$\frac{\partial L}{\partial w_1} = (-0.0400)(0.6) \approx -0.0240$$

$$\frac{\partial L}{\partial w_2} = (-0.0400)(0.1) \approx -0.0040$$

Dla biasu:

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1}$$

Ponieważ

$$\frac{\partial z_1}{\partial b_1} = 1$$

otrzymujemy

$$\frac{\partial L}{\partial b_1} = \delta_1 \approx -0.0400$$

Gradienty wag neuronu h_2

Analogicznie analizujemy neuron h_2:

$$z_2 = w_3 x_1 + w_4 x_2 + b_2$$

Stosujemy regułę łańcuchową:

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_3}$$

czyli

$$\frac{\partial L}{\partial w_3} = \delta_2 x_1$$

oraz

$$\frac{\partial L}{\partial w_4} = \delta_2 x_2$$

Po podstawieniu:

$$\frac{\partial L}{\partial w_3} = (0.0533)(0.6) \approx 0.0320$$

$$\frac{\partial L}{\partial w_4} = (0.0533)(0.1) \approx 0.0053$$

Dla biasu:

$$\frac{\partial L}{\partial b_2} = \delta_2 \approx 0.0533$$

Ważna obserwacja

Gradient każdej wagi w sieci neuronowej ma tę samą strukturę:

$$\frac{\partial L}{\partial w} = (\text{błąd neuronu}) \times (\text{wejście do tej wagi})$$

To właśnie dlatego backpropagation jest obliczeniowo bardzo efektywny — po policzeniu sygnałów błędu δ wszystkie gradienty można obliczyć bardzo prostymi mnożeniami.

7 Aktualizacja wag

Załóżmy współczynnik uczenia:

$$\eta = 0.1$$

Wtedy każdą wagę aktualizujemy według wzoru:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Na przykład dla w_5:

$$w_5^{\text{new}} = 0.3 - 0.1(-0.3509) = 0.3351$$

Dla w_6:

$$w_6^{\text{new}} = -0.4 - 0.1(-0.2150) = -0.3785$$

Podobnie aktualizujemy wszystkie pozostałe wagi i biasy.

8 Intuicja

Model przewidział wartość

$$\hat{y} = 0.2341$$

podczas gdy prawdziwa wartość wynosi

$$y = 0.8$$

czyli model przewidział wartość zbyt małą.

Dlatego po aktualizacji wag chcemy zwiększyć wpływ tych połączeń, które pomagają podnieść wynik wyjściowy, oraz zmniejszyć wpływ tych, które wynik obniżają.

Backpropagation robi to automatycznie:

- najpierw liczy błąd na wyjściu,
- potem przenosi ten błąd wstecz przez sieć,
- a następnie oblicza, jak poprawić każdą wagę.

9 Podsumowanie algorytmu

Cały proces uczenia dla jednego przykładu można zapisać w trzech krokach.

9.1 Forward pass

- oblicz sumy ważone,
- zastosuj funkcje aktywacji,
- oblicz wyjście sieci,
- oblicz funkcję straty.

9.2 Backward pass

- oblicz błąd na wyjściu,
- propaguj błąd wstecz przez sieć,
- oblicz gradienty wszystkich wag i biasów.

9.3 Gradient descent

- zaktualizuj wagi i biasy,
- powtarzaj proces dla kolejnych przykładów.

Backpropagation jest więc w istocie wielokrotnym zastosowaniem **reguły łańcuchowej** rachunku różniczkowego.